



## Práctica 6 – Programación Dinámica

Los objetivos de esta práctica son:

- Aprender a identificar subproblemas que están siendo calculados más de una vez en backtracking.
- Aprender a memoizar estos resultados en tablas de memorización.
- Aprender a diseñar algoritmos eficientes usando programación dinámica top-down y bottom-up.
- Reconstrucción de solución a partir de la tabla resultante del algoritmo.
- Demostración de correctitud (que va a ser muy parecida a los de backtracking).
- Demostración de complejidad, y comparación con algoritmos de backtracking.

Los ejercicios marcados con el símbolo ★ constituyen un subconjunto mínimo de ejercitación. Sin embargo, aconsejamos fuertemente hacer todos los ejercicios.

Los ejercicios marcados con el símbolo ⓘ tienen una ayuda al final de la guía.

---

### Ejercicio 1 (*El ejército del rey*) ★

El rey Cambyses está interesado en armar ejércitos en una serie de días consecutivos. Más aún, le interesa que el número de personas de su ejército en el día  $i \in \mathbb{N}$  sea equivalente a la suma del número de personas del ejército que formó el día  $i - 1$  e  $i - 2$ . La excepción para esto es en los días 0 y 1, en los cuales su ejército tendrá 1 sola persona. Para él es muy complicado determinar este número, así que nos pidió que lo ayudemos. Dado un día  $n \in \mathbb{N}$ , tenemos que devolver el número de personas de su ejército.

- Escribir la función recursiva que quiere calcular el rey en términos de  $n \in \mathbb{N}$ . ¿Es posible que esta función devuelva distintos valores para el mismo  $n$ ?
- ¿Qué complejidad tiene esta función si se la implementa de forma directa?
- Dibujar el árbol de llamadas recursivas para el día 4. ¿Se calculan las mismas cosas más de una vez? ¿Cuáles?
- Implementar un algoritmo recursivo en el lenguaje que prefieran que evite calcular dos veces lo mismo, guardando los resultados ya calculados para utilizarlos posteriormente. ⓘ
- Determinar la complejidad temporal y espacial de este nuevo algoritmo. ¿Cómo se compara con el algoritmo del Item b)?
- Optimizar el algoritmo anterior utilizando un enfoque *bottom-up* para lograr una complejidad espacial  $O(1)$ .

### Ejercicio 2 (*Vacaciones*) ★

Tomás tiene  $n \in \mathbb{N}$  días de vacaciones en los que puede hacer actividades.

- Se pueden hacer 2 actividades: gimnasio y competencias.
- Cada día puede tener disponible ninguna, alguna o ambas.

Tomás cada día puede o

- hacer una actividad que esté disponible, siempre que no la haya hecho el día anterior, o
- descansar.



Tomás tiene su calendario con su disponibilidad de actividades para cada día, y quiere minimizar los días de descanso. Nos pide que hagamos un algoritmo que determine cuál es la mínima cantidad de días de descanso que puede tener.

### Ejemplo:

- $n = 4$
- Días con gimnasio disponible:  $\{2, 3\}$
- Días con competencias disponibles:  $\{1, 2\}$

Puede lograr tener solo 2 días de descanso.

- Definir el universo de soluciones candidatas  $Sols$ , el predicado de validación  $válida_n(a)$  para una solución  $a \in Sols$ , y la relación de orden  $\leq$  entre soluciones válidas, de la misma manera que se hizo para los ejercicios de la guía de backtracking.
- Escribir el resultado que necesita calcular Tomás en términos de estos conjuntos de soluciones.
- Definir el conjunto de soluciones parciales  $Sols_{parciales}$ .
- Escribir una función recursiva que calcule la fórmula escrita en el Item b). ¿Qué argumentos hay que pasarle a la función para que calcule lo pedido? Demostrar su correctitud, poniéndole una semántica y demostrando que la cumple.
- Determinar la complejidad de implementar la función del Item d) de forma directa con backtracking.
- Diseñar un algoritmo que resuelva el problema que use memorización para alcanzar una complejidad temporal de  $O(n)$ . Comparar con el algoritmo de backtracking.
- Probar la complejidad temporal y espacial del algoritmo.
- Optimizar el algoritmo anterior utilizando un enfoque *bottom-up* para lograr una complejidad espacial  $O(1)$ .
- Implementar el algoritmo en su lenguaje de programación favorito.
- Modificar el algoritmo para que además de devolver la cantidad mínima de días de descanso, también devuelva una solución óptima. Es decir, indicarle a Tomás qué tiene que hacer cada día para poder alcanzar el mínimo número de días de descanso.

### Ejercicio 3 (Suma Subconjuntos: programación dinámica) ★

Recordemos el problema de suma de subconjuntos:

#### Problema: Suma Subconjuntos

Dado un multiconjunto  $C := \{c_1, \dots, c_n\}$  de números naturales y un natural  $k$ , queremos determinar si existe un subconjunto de  $C$  cuya sumatoria sea  $k$ . Vamos a suponer que  $C$  está ordenado de alguna forma arbitraria pero conocida (i.e.,  $C$  está implementado como la secuencia  $c_1, \dots, c_n$  o, análogamente, tenemos un iterador de  $C$ ).

- Sea  $n := |C|$  la cantidad de elementos de  $C$ . Considerar la siguiente función recursiva  $ss'_C: \{0, \dots, n\} \times \{0, \dots, k\} \rightarrow \text{Bool}$  tal que:

$$ss'_C(i, j) = \begin{cases} j = 0 & \text{si } i = 0 \\ ss'_C(i - 1, j) & \text{si } i \neq 0 \wedge C[i] > j \\ ss'_C(i - 1, j) \vee ss'_C(i - 1, j - C[i]) & \text{si no} \end{cases}$$



Convencerse de que esta es una definición equivalente de la función  $ss_{rec}$  del ?? de la guía de backtracking con la poda del ??, observando que  $ss_{rec}(C, k) = ss'_C(n, k)$ . En otras palabras, convencerse de que el algoritmo del ?? de la guía de backtracking es una implementación por backtracking de la función  $ss'_C$ . Concluir, pues, que  $\mathcal{O}(2^n)$  llamadas recursivas de  $ss'_C$  son suficientes para resolver el problema. ¿Cuál es la semántica de esta función?

- Observar que, como  $C$  no cambia entre llamadas recursivas, existen  $\mathcal{O}(nk)$  posibles entradas para  $ss'_C$ . Concluir que si  $k$  es mucho menor a  $2^n/n$  necesariamente algunas instancias de  $ss'_C$  son calculadas muchas veces por el algoritmo del ?? de la guía de backtracking. Mostrar un ejemplo donde se calcule varias veces la misma instancia.
- Considerar la estructura de memoización (i.e., el diccionario)  $M$  implementada como una matriz de  $(n + 1) \times (k + 1)$  tal que  $M[i, j]$  o bien tiene un valor indefinido  $\perp$  o bien tiene el valor  $ss'_C(i, j)$ , para todo  $0 \leq i \leq n$  y  $0 \leq j \leq k$ . Convencerse de que el Algoritmo 1 implementado de forma *top-down* mantiene un estado válido para  $M$  y computa  $M[i, j] = ss'_C(i, j)$  cuando se invoca  $ss'_C(i, j)$ .
- Concluir que `subset_sum_pd_top_down`( $C, n, k$ ) resuelve el problema. Calcular la complejidad temporal y espacial y compararla con el algoritmo `subset_sum` del ?? de la guía de backtracking. ¿Cuál algoritmo es mejor cuando  $k \ll 2^n/n$ ? ¿Y cuando  $k \gg 2^n/n$ ?
- Supongamos que queremos computar todos los valores de  $M$ . Una vez computados, por definición, obtenemos que

$$M[i, j] \stackrel{\text{def}}{=} ss'_C(i, j) \stackrel{ss'}{=} ss'_C(i - 1, j) \vee ss'_C(i - 1, j - C[i]) \stackrel{\text{def}}{=} M[i - 1, j] \vee M[i - 1, j - C[i]]$$

cuando  $i > 0$ , asumiendo que  $M[i - 1, j - C[i]]$  es falso cuando  $j - C[i] < 0$ . Por otra parte,  $M[0, 0]$  es verdadero, mientras que  $M[0, j]$  es falso para  $j > 0$ . A partir de esta observación, concluir que el Algoritmo 2 implementado de forma *bottom-up* computa  $M$  correctamente y, por lo tanto,  $M[i, j]$  contiene la respuesta al problema de la suma para todo  $\{c_1, \dots, c_i\}$  y  $j$ .

- Modificar el algoritmo *bottom-up* anterior para mejorar su complejidad espacial a  $\mathcal{O}(k)$ .

---

#### Algoritmo 1 Top-down

---

```

1 Inicializar  $M[i, j] \leftarrow \perp$  para todo  $0 \leq i \leq n$  y  $0 \leq j \leq k$ 
2 function subset_sum_pd_top_down( $C, i, j$ )
3   if  $j < 0$  then
4     return falso
5   if  $i = 0$  then
6     return ( $j = 0$ )
7   if  $M[i, j] = \perp$  then
8      $M[i, j] \leftarrow$  subset_sum_pd_top_down( $C, i - 1, j$ )  $\vee$  subset_sum_pd_top_down( $C, i - 1, j - C[i]$ )
9   return  $M[i, j]$ 
```

---



---

#### Algoritmo 2 Bottom-up

---

```

1 function subset_sum_pd_bottom_up( $C, k$ )
2   Inicializar  $M[0, j] \leftarrow (j = 0)$  para todo  $0 \leq j \leq k$ 
3   for  $i = 1$  to  $n$  do
4     for  $j = 0$  to  $k$  do
5        $M[i, j] \leftarrow M[i - 1, j] \vee (j - C[i] \geq 0 \wedge M[i - 1, j - C[i]])$ 
```

---

#### Ejercicio 4 (Contar caminos en DAGs) ★

- Proponer un algoritmo que tome tiempo  $\mathcal{O}(n + m)$  para contar la cantidad de caminos distintos en un DAG entre dos vértices  $s$  y  $t$ . Demostrar su correctitud y complejidades temporal y espacial. **i**



- b) Si lo hicieron top-down, háganlo bottom-up, y viceversa. 

### Ejercicio 5 (*OptiPago*) ★

Tenemos un multiconjunto  $B$  de valores de billetes y queremos comprar un producto de costo  $c$  de una máquina que no da vuelto. Para poder adquirir el producto debemos cubrir su costo usando un subconjunto de nuestros billetes. El objetivo es pagar con el mínimo exceso posible a fin de minimizar nuestra pérdida. Más aún, queremos gastar el menor tiempo posible poniendo billetes en la máquina. Por lo tanto, entre las opciones de mínimo exceso posible, queremos una con la menor cantidad de billetes. Por ejemplo, si  $c = 14$  y  $B = \{2, 3, 5, 10, 20, 20\}$ , la solución es pagar 15, con exceso 1, insertando solo dos billetes: uno de 10 y otro de 5.

Considerar la siguiente estrategia por *backtracking* para el problema, donde  $B = \{b_1, \dots, b_n\}$ . Tenemos dos posibilidades: o agregamos el billete  $b_n$ , gastando un billete y quedando por pagar  $c - b_n$ , o no agregamos el billete  $b_n$ , gastando 0 billetes y quedando por pagar  $c$ .

- Definir los conjuntos de soluciones candidatas y válidas, y escribir la fórmula que corresponde a la solución al problema.
- Definir el conjunto de soluciones parciales. Escribir una función recursiva  $\text{OptiPago}(B, c)$  para resolver el problema, donde  $\text{OptiPago}(B, c) = (c', q)$  es el mínimo costo mayor o igual a  $c$  que es posible pagar con los billetes de  $B$  es  $c'$  y la cantidad de billetes mínima es  $q$ . Indicar cuál es la llamada a  $\text{OptiPago}(B, c)$  que resuelve nuestro problema y demostrar por inducción que la función  $\text{OptiPago}$  es correcta.
- Implementar la función del Item [b](#)) en un lenguaje de programación imperativo utilizando una función recursiva con parámetros  $(B, i, j)$  que compute  $\text{OptiPago}(\{b_1, \dots, b_i\}, j)$ . ¿Cuál es la complejidad del algoritmo?
- Reescribir  $\text{OptiPago}$  como una función recursiva  $\text{OptiPago}'_B(i, j) = \text{OptiPago}(\{b_1, \dots, b_i\}, j)$  que implemente la idea anterior **dejando fijo el parámetro  $B$** . Determinar cuándo es mejor implementar  $\text{OptiPago}'_B$  con memoización y cuándo es mejor sin en términos de complejidad temporal.
- Adaptar el algoritmo del Item [c](#)) para incluir la estructura de memoización.
- Escribir un algoritmo *bottom-up* para calcular todos los valores de la estructura de memoización y discutir cómo se puede reducir la memoria extra consumida por el algoritmo.

### Ejercicio 6 (*AstroTrade*) ★

Astro Void se dedica a la compra de asteroides. Sea  $p \in \mathbb{N}^n$  tal que  $p_i$  es el precio de un asteroide el  $i$ -ésimo día en una secuencia de  $n$  días. Astro Void quiere comprar y vender asteroides durante esos  $n$  días de manera tal de obtener la mayor ganancia neta posible. Debido a las dificultades que existen en el transporte y almacenamiento de asteroides, Astro Void puede comprar a lo sumo un asteroide cada día, puede vender a lo sumo un asteroide cada día y comienza sin asteroides. Además, el Ente Regulador Asterooidal impide que Astro Void venda un asteroide que no haya comprado. Queremos encontrar la máxima ganancia neta que puede obtener Astro Void respetando las restricciones indicadas. Por ejemplo, si  $p = (3, 2, 5, 6)$  el resultado es 6 y si  $p = (3, 6, 10)$  el resultado es 7. Notar que en una solución óptima, Astro Void debe terminar sin asteroides.

- Escribir el enunciado en términos de los conjuntos de soluciones de siempre.
- Diseñar una función recursiva que resuelva el problema, dejando fijo el vector de precios  $p$ . Indicar qué dato es la respuesta al problema con esa formulación recursiva.
- Demostrar que su formulación recursiva es equivalente a la formalización realizada en el Item [a](#)).
- Diseñar un algoritmo de PD *top-down* que resuelva el problema y explicar su complejidad temporal y espacial auxiliar.
- Diseñar un algoritmo de PD *bottom-up*, reduciendo la complejidad espacial.



### Ejercicio 7 (*Cortes económicos*)

Debemos cortar una vara de madera en varios lugares predeterminados. Sabemos que el costo de realizar un corte en una madera de longitud  $\ell$  es  $\ell$  (y luego de realizar ese corte quedarán 2 varas de longitudes que sumarán  $\ell$ ). Por ejemplo, si tenemos una vara de longitud 10 metros que debe ser cortada a los 2, 4 y 7 metros desde un extremo, entonces los cortes se pueden realizar, entre otras maneras, de las siguientes formas:

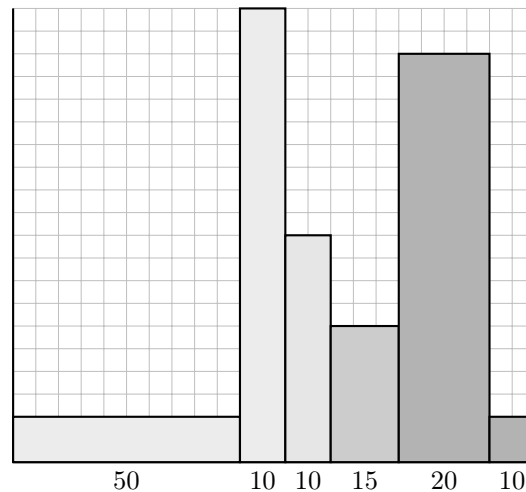
- Primero cortar en la posición 2, después en la 4 y después en la 7. Esta resulta en un costo de  $10 + 8 + 6 = 24$  porque el primer corte se hizo en una vara de longitud 10 metros, el segundo en una de 8 metros y el último en una de 6 metros.
- Cortar primero donde dice 4, después donde dice 2, y finalmente donde dice 7, con un costo de  $10 + 4 + 6 = 20$ , que es menor.

Queremos encontrar el mínimo costo posible de cortar una vara de longitud  $\ell$ .

- a) Convencerse de que el mínimo costo de cortar una vara que abarca desde  $i$  hasta  $j$  con el conjunto  $C$  de lugares de corte es  $j - i$  más el mínimo, para todo lugar de corte  $c$  entre  $i$  y  $j$ , de la suma entre el mínimo costo desde  $i$  hasta  $c$  y el mínimo costo desde  $c$  hasta  $j$ .
- b) Escribir matemáticamente una formulación recursiva basada en el Item a). Explicar su semántica e indicar cuáles serían los parámetros para resolver el problema. Demostrar su correctitud.
- c) Diseñar un algoritmo de PD y dar su complejidad temporal y espacial auxiliar. Comparar cómo resultaría un enfoque *top-down* con uno *bottom-up*.
- d) Supongamos que se ordenan los elementos de  $C$  en un vector *cortes* de menor a mayor y se agrega un 0 al principio y un  $\ell$  al final. Luego, en vez de considerar el costo entre dos cortes  $c_1$  y  $c_2$ , se considera el mínimo costo entre dos *índices*  $i$  y  $j$  del arreglo *cortes*.
  - I) Escribir una formulación recursiva con dos parámetros que esté basada en la idea del Item d) y explicar su semántica. Demostrar su correctitud.
  - II) Diseñar un algoritmo de PD, dar su complejidad temporal y espacial auxiliar, y compararlas con aquellas del Item c). Comparar cómo resultaría un enfoque *top-down* con uno *bottom-up*.

### Ejercicio 8 (*Mi Buenos Aires Crecido*)

Sasha vive en San Nicolás hace mucho tiempo, y tiene una eterna discusión con su vecina Tasha. Sasha dice que si se mira el horizonte de izquierda a derecha terminando en el obelisco, los edificios están ordenados en un perfil principalmente ascendente para que la altura del obelisco sea más impresionante. Tasha le dice que con las nuevas torres que se construyeron en la zona eso ya no es verdad, y que en realidad ahora menos de la mitad del ancho del horizonte está en orden ascendente.



En este ejemplo la máxima subsecuencia ascendente tiene 85 de ancho, usando 3 edificios.

- Los datos de entrada son los  $N$  edificios, con dos funciones  $alto(i)$  y  $ancho(i)$  que nos dan el alto y ancho del edificio  $i$ . Escribir una formulación recursiva que sea la base de un algoritmo de PD, que dados los parámetros  $act$  y  $ult$ , los cuales nos dicen el edificio actual y el último edificio de la subsecuencia hasta ahora, compute el ancho de la máxima subsecuencia ascendente. Dar su complejidad temporal y espacial auxiliar, y demostrar su correctitud.
- Utilizar la función del inciso anterior para ver quien tiene razón entre Sasha o Tasha.
- Resolver el problema con una complejidad espacial auxiliar de  $O(n)$  y con la misma complejidad temporal utilizando un enfoque *bottom-up*.
- (Difícil) Resolver el problema con una complejidad temporal de  $O(N \cdot \log N)$ .

### Ejercicio 9 (Travesía vital)

Hay un terreno, que podemos pensarlo como una grilla de  $m$  filas y  $n$  columnas, con trampas y pociones. Queremos llegar de la esquina superior izquierda hasta la inferior derecha, y desde cada casilla solo podemos movernos a la casilla de la derecha o a la de abajo. Cada casilla  $(i, j)$  tiene un número entero  $A_{i,j}$  que nos modificará el nivel de vida sumándonos el número  $A_{i,j}$  (si es negativo, nos va a restar  $|A_{i,j}|$  de vida). Queremos saber el mínimo nivel de vida con el que debemos comenzar tal que haya un camino posible de modo que en todo momento nuestro nivel de vida sea al menos 1. Por ejemplo, si tenemos la grilla

$$A = \begin{bmatrix} -2 & -3 & 3 \\ -5 & -10 & 1 \\ 10 & 30 & -5 \end{bmatrix},$$

el mínimo nivel de vida con el que podemos comenzar es 7 porque podemos realizar el camino que va todo a la derecha y todo abajo.

- Pensar la idea de un algoritmo de *backtracking* (no hace falta escribirlo).
- Demostrar que, excepto que estemos en los límites del terreno, la mínima vida necesaria al llegar a la posición  $(i, j)$  es el resultado de restar al mínimo entre la mínima vida necesaria en  $(i + 1, j)$  y aquella en  $(i, j + 1)$ , el valor  $A_{i,j}$ , salvo que eso fuera menor o igual que 0, en cuyo caso sería 1.
- Escribir una formulación recursiva basada en [b\)](#). Explicar su semántica e indicar cuáles serían los parámetros para resolver el problema.



- d) Diseñar un algoritmo de PD y dar su complejidad temporal y espacial auxiliar. Comparar cómo resultaría un enfoque *top-down* con uno *bottom-up*.
- e) Dar un algoritmo *bottom-up* cuya complejidad temporal sea  $\mathcal{O}(m \cdot n)$  y la espacial auxiliar sea  $\mathcal{O}(\min(m, n))$ .

### Ejercicio 10 (Apilando cajas)

Tenemos cajas numeradas de 1 a  $N$ , todas de iguales dimensiones. Queremos encontrar la máxima cantidad de cajas que pueden apilarse en una única pila cumpliendo que:

- solo puede haber una caja apoyada directamente sobre otra;
- las cajas de la pila deben estar ordenadas crecientemente por número, de abajo para arriba;
- cada caja  $i$  tiene un peso  $w_i$  y un soporte  $s_i$ , y el peso total de las cajas que están arriba de otra no debe exceder el soporte de esa otra.

Si tenemos los pesos  $w = [19, 7, 5, 6, 1]$  y los soportes  $s = [15, 13, 7, 8, 2]$  (la caja 1 tiene peso 19 y soporte 15, la caja 2 tiene peso 7 y soporte 13, etc.), entonces la respuesta es 4. Por ejemplo, pueden apilarse de la forma 1-2-3-5 o 1-2-4-5 (donde la izquierda es más abajo), entre otras opciones.

- a) Pensar la idea de un algoritmo de *backtracking* (no hace falta escribirlo).
- b) Escribir una formulación recursiva que sea la base de un algoritmo de PD. Explicar su semántica e indicar cuáles serían los parámetros para resolver el problema. Formalizar el problema y demostrar la correctitud de la función recursiva.
- c) Diseñar un algoritmo de PD y dar su complejidad temporal y espacial auxiliar. Comparar cómo resultaría un enfoque *top-down* con uno *bottom-up*.

### Ejercicio 11 (Operaciones)

Sea  $v = (v_1, v_2, \dots, v_n)$  un vector de números naturales, y sea  $w \in \mathbb{N}$ . Se desea intercalar entre los elementos de  $v$  las operaciones  $+$  (suma),  $\times$  (multiplicación) y  $\uparrow$  (potenciación) de tal manera que al evaluar la expresión obtenida el resultado sea  $w$ . Para evaluar la expresión se opera de izquierda a derecha ignorando la precedencia de los operadores. Por ejemplo, si  $v = (3, 1, 5, 2, 1)$ , y las operaciones elegidas son  $+$ ,  $\times$ ,  $\uparrow$  y  $\times$  (en ese orden), la expresión obtenida es  $3 + 1 \times 5 \uparrow 2 \times 1$ , que se evalúa como  $((3 + 1) \times 5) \uparrow 2 \times 1 = 400$ .

- a) Escribir una formulación recursiva que sea la base de un algoritmo de PD que, dados  $v$  y  $w$ , encuentre una secuencia de operaciones como la deseada y la devuelva, en caso de que tal secuencia exista. Explicar su semántica e indicar cuáles serían los parámetros para resolver el problema. Formalizar el problema y demostrar que la función recursiva es correcta.
- b) Diseñar un algoritmo basado en PD con la formulación del Item a) y dar su complejidad temporal y espacial auxiliar. Comparar cómo resultaría un enfoque *top-down* con uno *bottom-up*.

### Ejercicio 12 (Suma de dados)

Se arrojan simultáneamente  $n$  dados, cada uno con  $k$  caras numeradas de 1 a  $k$ . Queremos calcular todas las maneras posibles de conseguir la suma total  $s \in \mathbb{N}$  con una sola tirada. Tomamos dos variantes de este problema.

- (A) Consideramos que los dados son **distinguibiles**, es decir que si  $n = 3$  y  $k = 4$ , entonces existen 10 posibilidades que suman  $s = 6$ :

- a) 4 posibilidades en las que el primer dado vale 1: , , , .
- b) 3 posibilidades en las que el primer dado vale 2: , , .



- c) 2 posibilidades en las que el primer dado vale 3:  $\begin{smallmatrix} \blacksquare & \blacksquare & \blacksquare \\ \blacksquare & \blacksquare & \blacksquare \end{smallmatrix}$ ,  $\begin{smallmatrix} \blacksquare & \blacksquare & \blacksquare \\ \blacksquare & \blacksquare & \blacksquare \end{smallmatrix}$ .
- d) Una posibilidad en la que el primer dado vale 4:  $\begin{smallmatrix} \blacksquare & \blacksquare & \blacksquare \\ \blacksquare & \blacksquare & \blacksquare \end{smallmatrix}$ .
- (B) Consideramos que los dados son **indistinguibles**, es decir que si  $n = 3$  y  $k = 4$ , entonces existen 3 posibilidades que suman  $s = 6$ :
- a) Un dado vale 4, los otros dos valen 1.
- b) Un dado vale 3, otro 2 y otro 1.
- c) Todos los dados valen 2.
- a) Definir en forma recursiva la función  $f: \mathbb{N}^2 \rightarrow \mathbb{N}$  tal que  $f(n, s)$  devuelve la respuesta para el escenario (A) (fijado  $k$ ).
- b) Definir en forma recursiva la función  $g: \mathbb{N}^3 \rightarrow \mathbb{N}$  tal que  $g(n, s, k)$  devuelve la respuesta para el escenario (B).
- c) Definir algoritmos de PD top-down para calcular  $f(n, s)$  y  $g(n, s, k)$  indicando claramente las estructuras de datos utilizadas y la complejidad resultante.
- d) Comparar las complejidades temporales y espaciales con la implementación con backtracking de las funciones  $f$  y  $g$ .
- e) Escribir el (pseudo-)código de los algoritmos top-down resultantes.

**Nota:** Una solución correcta de este ejercicio debería indicar cómo se computa tanto  $f(n, s)$  como  $g(n, s, k)$  en tiempo  $O(nk \min\{s, nk\})$ .

### Ejercicio 13 (Las legiones del César)

Este problema fue extraído de [Codeforces \(118D\)](#).

Al famoso general César le gusta poner en fila sus tropas, que son patos y dodos.

- Tiene un ejército de  $P$  patos y  $D$  dodos.
- Quiere que la fila cumpla que:
  - No puede haber más de  $m_P$  patos consecutivos.
  - No puede haber más de  $m_D$  dodos consecutivos.
- Los patos son indistinguibles entre ellos, al igual que los dodos.

Le interesa contar la cantidad de formas posibles de formar esta fila.

- a) Definir una función recursiva  $f_{m_P, m_D}(t, n_P, n_D, k_P, k_D)$  que calcule la cantidad de formas de poner los patos y dodos restantes, dado que:
- Quedan  $t$  lugares disponibles en la fila.
  - Quedan  $n_P$  patos y  $n_D$  dodos disponibles.
  - Al final de la fila hay, actualmente,  $k_P$  patos y  $k_D$  dodos formados de forma consecutiva.

Pensar qué elección tomamos en cada paso.

- b) Pensar cómo modificar la función para que su firma sea  $f_{m_P, m_D}(n_P, n_D, k, \text{ultTropa})$ , donde  $\text{ultTropa}$  nos indica cuál es la última tropa puesta y  $k$  la cantidad consecutiva de la misma.
- c) Pensar cómo modificar la función para que su firma sea  $f_{m_P, m_D}(n_P, n_D, \text{ultTropa})$ .
- d) Calcular la complejidad temporal y espacial de cada una de las versiones de la función.
- e) Demostrar la correctitud de cada una de las versiones de la función.
- f) **(Difícil)** Pensar cómo optimizar la solución para que sea  $O(P \cdot D)$  en tiempo y espacio (enfoque *bottom-up*).

#### Ejercicio 14 (*Granjero*)

Este problema fue extraído de [Codeforces \(41D\)](#).

Un granjero tiene un terreno de  $N$  metros de largo y  $M$  de ancho, dividido en celdas de  $1\text{m} \times 1\text{m}$ . En algunas celdas hay una cantidad de arvejas definida previamente. El granjero tiene como objetivo recolectar la mayor cantidad posible de arvejas, respetando que:

- Empieza desde alguna celda en el comienzo del terreno ( $y = 0$ ).
- Se puede mover en 2 sentidos: **adelante e izquierda** y **adelante y derecha**.
- Tiene que llegar al final del terreno ( $y = N$ ) con una cantidad de arvejas recolectadas divisible por  $K + 1$ , con  $K$  un número fijo dado.

Diseñar un algoritmo que en  $O(N \cdot M \cdot K)$  calcule cuál es la máxima cantidad de arvejas que puede recolectar el granjero. Demostrar su correctitud y complejidad.

Indicar también como es posible reconstruir un posible camino para recolectar esa cantidad de arvejas.

Comparar cómo resultaría un enfoque *top-down* con uno *bottom-up*.

## Ayudas

b) Para el bottom-up, computar  $d$  de acuerdo a un orden topológico  $v = v_1, \dots, v_n$  donde  $v_i \rightarrow v_j$  solo si  $i < j$ . Este orden se puede computar en  $O(n + m)$  (ver guía de algoritmos sobre grafos).

### Ejercicio 4

a) Considerar que su función recursiva calcule la cantidad de caminos distintos que terminan en un vértice dado del DAG.

### Ejercicio 4

Considerar guardar los valores ya calculados en un arreglo de  $n$  posiciones, y acceder a ese arreglo para ver si el valor actual ya fue calculado o no.

### Ejercicio 1